

GRA-Project A207

Image Processing

Clemens März, Lucas Michel, Justus Pansch

August 23, 2024



TUM Uhrenturm

- 1 Problem Statement
- 2 Solution Approach and Design Decisions
- 3 Optimizations
- 4 Correctness and Accuracy
- 5 Performance Analysis

Problem Statement

- Convert color values to grayscale values using a weighted average

$$q = \frac{a * r + b * g + c * b}{a + b + c}$$

- Brightness adjustment

$$q' = clamp_0^{255}(q + brightness)$$

- Calculation of the mean and standard deviation

$$\mu = \frac{1}{|Q|} \sum_{q \in Q} q' \qquad \sigma = \sqrt{\frac{1}{|D|} \sum_{q \in Q} (q' - \mu)^2}$$

Problem Statement

- Contrast adjustment

$$q'' = clamp_0^{255} \left(\frac{k}{\sigma} * q' + \left(1 - \frac{k}{\sigma}\right) * \mu \right)$$

- Method for square root calculation using basic operations
- Determination of reasonable default values for the weighted average

Outline

- 1 Problem Statement
- 2 Solution Approach and Design Decisions
 - Framework Program
 - Grayscale Conversion/Brightness Adjustment
 - Contrast Adjustment
- 3 Optimizations
- 4 Correctness and Accuracy
- 5 Performance Analysis

Input File Format

■ Portable Pixel Map (ppm)¹

■ Structure

- ☐ Magic Number: P6
- ☐ Image size: Two numbers specifying width and height in pixels
- ☐ Maximum color value: Defines color depth
- ☐ Pixel data: Binary data, each pixel represented by three bytes (Red, Green, Blue)

■ Special features of the file format

- ☐ Multiple images in one file possible
- ☐ Comments in the header allowed
- ☐ Maximum color value may be less than 255

¹<https://netpbm.sourceforge.net/doc/ppm.html>

Output File Format

■ Portable Gray Map (pgm)²

■ Structure

- ☐ Magic Number: P5
- ☐ Image size: Two numbers specifying width and height in pixels
- ☐ Maximum gray value: defines grayscale depth
- ☐ Pixel data: Binary data, each pixel represented by one byte

■ Advantages

- ☐ More compact than PPM since only one byte per pixel is needed

²<https://netpbm.sourceforge.net/doc/pgm.html>

Default Values for Weights a, b and c

- Equal weighting of all color components ($a = b = c$)
- Weighting according to human perception³: The human eye perceives green more strongly than red and red more strongly than blue

- ☐ Resulting weighting

$$D = 0.299 * R + 0.587 * G + 0.114 * B$$

- ☐ Based on the composition of the retina, especially the sensitivity of the cones

³https://www.itu.int/dms_pubrec/itu-r/rec/bt/R-REC-BT.601-7-201103-I!!PDF-E.pdf

Special Case: $a = b = c = 0$

$$D = \frac{a * R + b * G + c * B}{a + b + c}$$

- Division by zero $\rightarrow$ mathematically undefined
- Possible application: Set all pixels to a specific value using `--brightness`

Special Case: Negative Weights

■ Options:

- ☐ Absolute value
- ☐ Invalid input

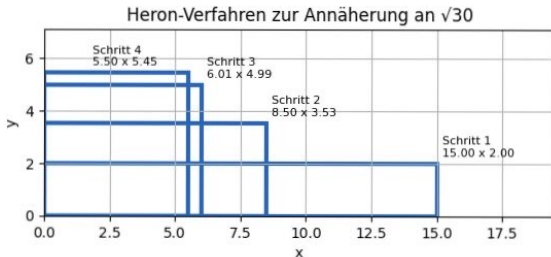
$$D = \frac{a * R + b * G + c * B}{a + b + c}$$

- Negative parameters may lead to $a + b + c = 0$ with $a, b, c \neq 0 \rightarrow$ Undefined
- Both negative and excessively large values can occur $\rightarrow$ weighted average is invalid
- No extended functionality

Square Root Calculation Using Basic Operations

Heron Method

- We use the Heron method⁴ to approximate square roots
- Geometrically, iteratively adjust the sides of a rectangle $\rightarrow$ square $\rightarrow$ side length is the root of the area
- $x' = \frac{1}{2} * (x + \frac{A}{x})$



⁴<https://de.wikipedia.org/wiki/Heron-Verfahren>

Special Case: Standard Deviation $\sigma = 0$

$$\frac{k}{\sigma} * q + (1 - \frac{k}{\sigma}) * \mu$$

- Mathematically undefined for $\sigma = 0$
- Practically not computable for $\sigma \approx 0$
- $\sigma = 0 \rightarrow$ all pixels have the same color, i.e., $\forall q \in Q : q = \mu$
- Contrast adjustment is based on deviation from the mean $\rightarrow$ contrast cannot be adjusted
- Each pixel keeps its value or is overwritten with the mean

Outline

- 1 Problem Statement
- 2 Solution Approach and Design Decisions
- 3 Optimizations**
 - SIMD
 - Mathematical Transformation of Standard Deviation
 - Precomputation
 - Fixed-Point Arithmetic
- 4 Correctness and Accuracy
- 5 Performance Analysis

Applicability of SIMD

- Data parallelism: pixels can be processed independently
- Common operations: operations are identical for all pixels
- Continuous memory access: data is stored consecutively and can be loaded/stored efficiently
 - Parallelization with SIMD increases processing speed

Standard Deviation σ - Transformation

- Rearranging the formula for σ^2 allows the sum to be computed only with integers

$$\begin{aligned}\sigma^2 &= \frac{1}{|D|} \sum_{q \in Q} (q - \mu)^2 = \frac{1}{|D|} \sum_{q \in Q} (q^2 - 2q\mu + \mu^2) \\ &= \frac{1}{|D|} \left(\sum_{q \in Q} q^2 - 2\mu \sum_{q \in Q} q + |D|\mu^2 \right)\end{aligned}$$

- Q represents the set of pixel values

Standard Deviation σ - Advantages

$$\sigma^2 = \frac{1}{|D|} \left(\sum_{q \in Q} q^2 - 2\mu \sum_{q \in Q} q + |D|\mu^2 \right)$$

- Pixel values $q \in Q$ have type `uint8_t`
- Advantages:
 - SIMD: higher parallelizability since more values fit in one vector
 - Arithmetic operations inside the loop are faster

Squares in Standard Deviation

$$\sigma^2 = \frac{1}{|D|} \left(\sum_{q \in Q} q^2 - 2\mu \sum_{q \in Q} q + |D|\mu^2 \right)$$

- Pixel values $q \in Q$ have type `uint8_t`
- Store squares in an array
- Array access potentially faster than squaring
- Operations inside the summation more performant

Fixed-Point Numbers

- Fixed-point arithmetic using integer types
- Conversion between fixed-point and floating-point numbers
 - Define the number of fractional bits n
 - Multiply floating-point numbers by 2^n and store in integers
 - Perform arithmetic
 - Convert back to integers (via right shift by n and rounding if needed)
- Convert floating-point to fixed-point only outside loops
- Integer arithmetic inside loops

Contrast Adjustment - Fixed-Point SIMD

- Parallelized version of contrast adjustment uses floating-point numbers (k , σ , and μ are floating-point)
 - ☐ Slow due to limited parallelization
 - ☐ Slow due to floating-point arithmetic
 - Using 8.8 fixed-point numbers
 - ☐ Integer arithmetic
 - ☐ 16 instead of 32 bits per value
- higher speedup

$$\frac{k}{\sigma} * q + \left(1 - \frac{k}{\sigma}\right) * \mu = \frac{k}{\sigma} * q + \mu - \frac{k}{\sigma} * \mu = \frac{k}{\sigma} * q - \frac{k}{\sigma} * \mu + \mu = \frac{k}{\sigma} * \left(q - \mu + \frac{\sigma}{k} * \mu\right)$$

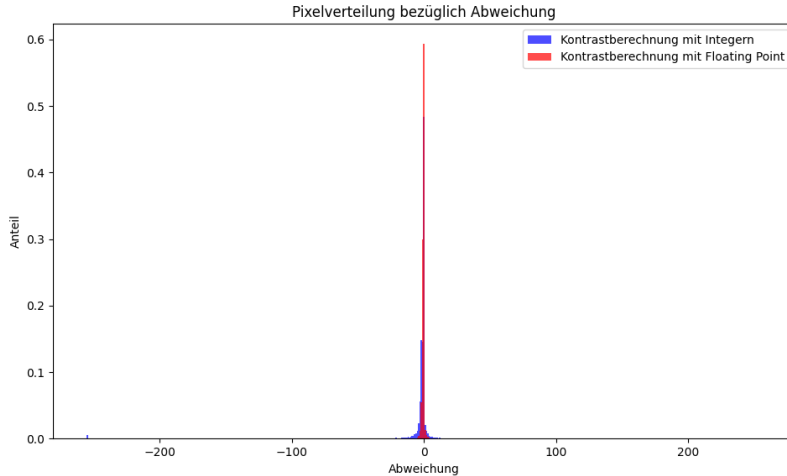
- Precompute $\mu - \frac{\sigma}{k} * \mu \rightarrow$ either addition or subtraction with integers depending on the case

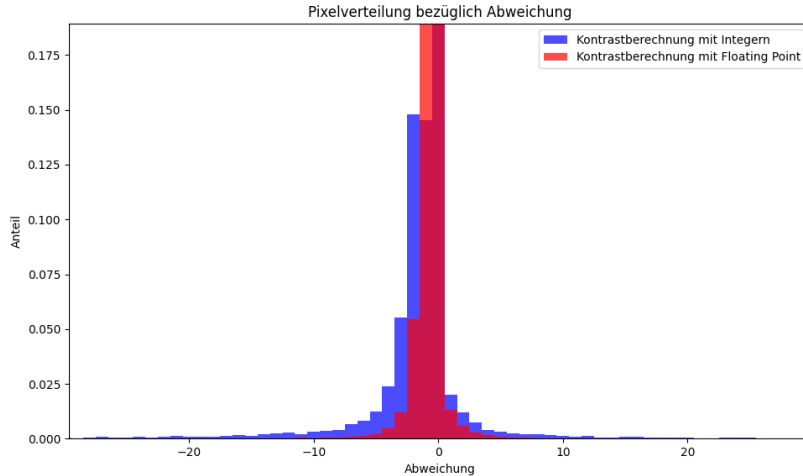
Fixed-Point Arithmetic

- No vectorization → fixed-point numbers can be stored in `uint64_t`, because `double` has 52 mantissa bits → 12.52 fixed-point
- 12.52 fixed-point → multiplication with `uint8_t` pixel values possible without overflow
- Shifts do not round → fast fixed-point rounding using bitmask
 - ☐ Determine integer part by shift
 - ☐ Isolate first fractional bit
 - ☐ Shift this bit to position zero
 - ☐ Add to integer part

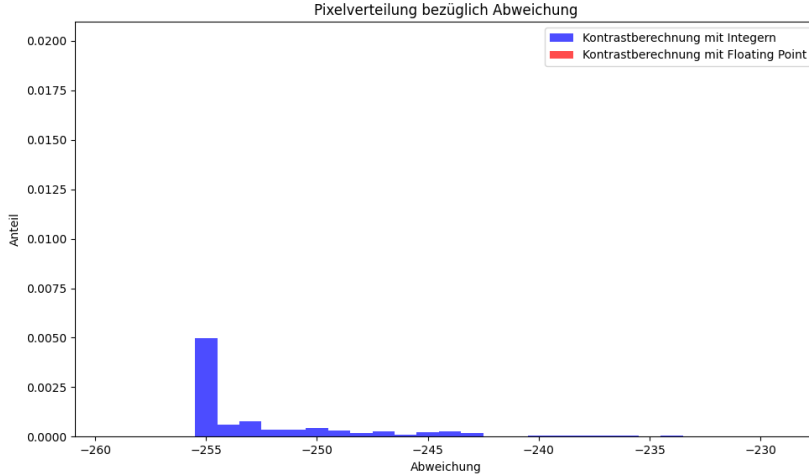
Outline

- 1 Problem Statement
- 2 Solution Approach and Design Decisions
- 3 Optimizations
- 4 Correctness and Accuracy**
- 5 Performance Analysis

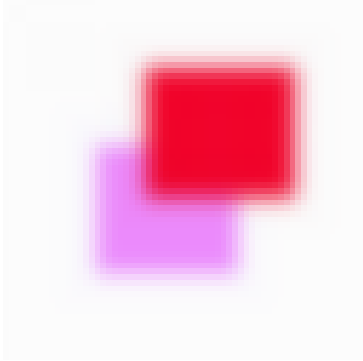




Deviation - Range at 255



Example - Deviations at Transitions



(a) Input



(b) Basic Implementation



(c) SSE with Floating-Point

Example - Small σ



(a) Input



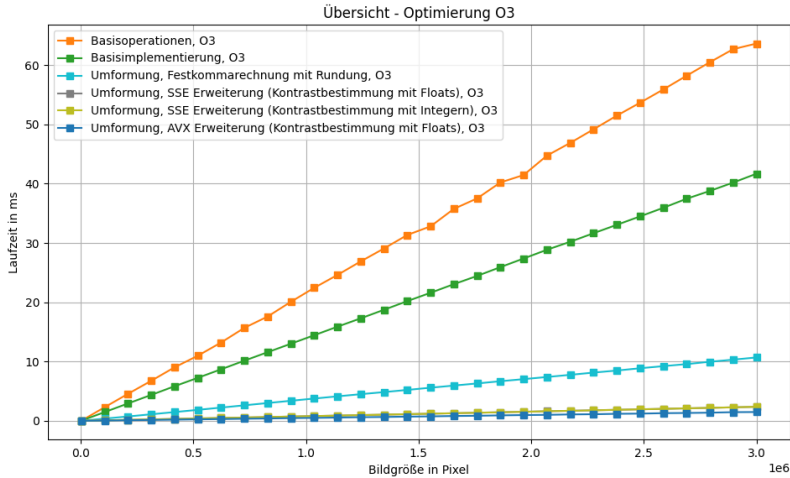
(b) Basic Implementation

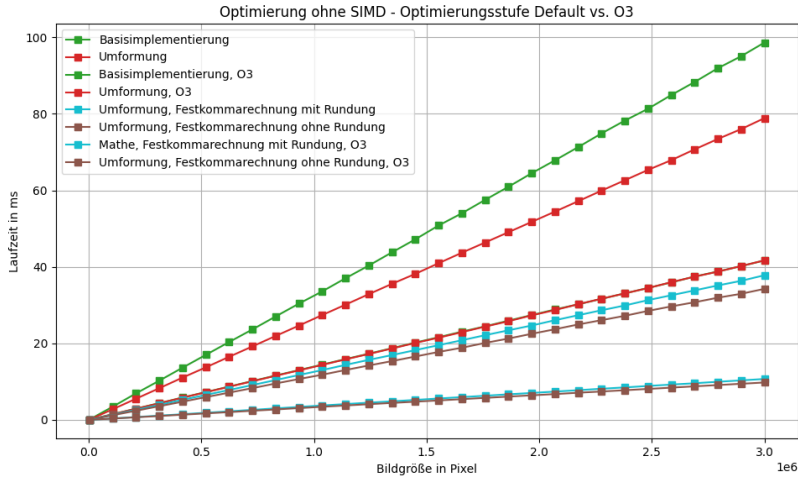


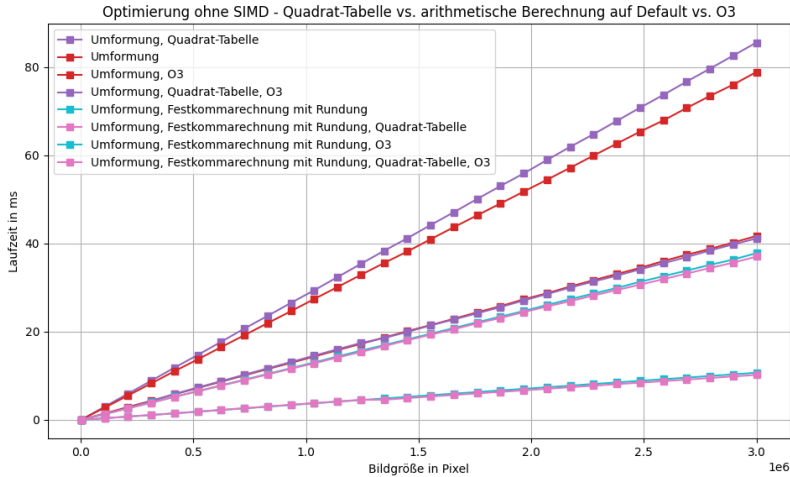
(c) AVX with Floating-Point

Outline

- 1 Problem Statement
- 2 Solution Approach and Design Decisions
- 3 Optimizations
- 4 Correctness and Accuracy
- 5 Performance Analysis**







SIMD - O3 vs O0

